

ADVANCED PROGRAMMING

OBJECT ORIENTED PROGRAMMING IN JAVA

Vicenç Soler

NORTHIN SUMMER SCHOOL

Barcelona, June 2026

Java Syntax

- Syntax identical to C for the usual operations: variable declaration, if, loops, function definition, castings (type conversions), etc.
 - Also the operators: ++, --, %, etc. and the same issue with integer division.
 - We avoid pointers definition (it is easier to manage it). Fields are only accessed via "." (no "->")
 - Structs are classes.
 - Functions and variables are not allowed to be declared outside classes.
 - No need to sort the functions depending on when it is called.

- Type of variables are:

- Integer numbers: **byte, short, int, long**
- Real numbers: **float, double**.
- Other types:
 - **char** (character, not an integer)
 - **boolean** (only values true and false).

- Arrays: index starts at 0:

int v[]=new int[5]; //declares an array of 5 integers

v[0]=32;

v[5]=40; //Java crashes in case we index outside the array range.

v.length -> returns the length of the array (5 in this case)



Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Java vs C

- An important difference between Java and C is how the programs are executed:
 - C creates an executable file that runs directly in the CPU.
 - Java does not create an executable file, it creates object files (one per defined class, with extension “.class”) that are interpreted by another program, named Java Virtual Machine (JVM).
 - This object files contain pseudo-compiled instructions that JVM interpret and execute.
- This causes Java programs to run slower than C programs, since every instruction is not executed directly on CPU, but executed by another software program.
- Java has the Garbage Collector process, that allows you to automatically free memory allocated when pointers are lost.
- The advantage of Java: once you have a program compiled you can run it on Windows, Linux or MacOS without needing to recompile again. C programs need to be recompiled when running in another OS.
- Its success in the industry is because it is faster to develop applications because it is easier to manage pointers and memory, reducing the number of bugs that may exist in a program.

Program example

//All the functions and variables must be inside classes. So, we must define the “main” function inside a class:

```
class Main {    //Any other name than Main can be used
    public static void main(String argv[]) {    //Syntax sensitive to upper/lowercases (as C)
        System.out.println("Hello world");    //Prints "Hello world", ending with "\n"
    }
}
```

- **main()** function:
 - As in C, it contains the main program.
 - Its syntax must be exactly this one. We are only allowed to change the name of the variable “argv”.
 - argv is an array of strings that contains the set of arguments the program may receive when run.

The main function: arguments

- Receiving arguments to a program:
- In C:

```
//argc=arguments counter: length of the argv
//argv=argument vector: array of strings
int main(int argc, char *argv[]) {
    int i;
    for(i=0;i<argc;i++) {
        printf("argument: %s\n",argv[i]);
    }
}
//In C, first argument is the program name
```

Running:

```
C\>myprogam summer 2020
myprogram
summer
2020
```

- In Java:

```
class Main {
    public static void main(String argv[]) {
        for(int i=0;i<argv.length;i++) {
            System.out.println("argument:"+argv[i]);
        }
    }
} //Close Main class
```

//In Java, the main function only needs 1 parameter since the length of the array is inside the variable.
//**argv.length** contains the length of the array "argv".

```
C\>java Main summer 2020
summer
2020
```

Object Oriented Programming: Classes & Objects

- A **class** is: a struct of C + functions . So it contains variables (**attributes**) and functions (**methods**).
- An **object** is the instance of a class: The class is the structure definition and every instance of that class (creation of the space to store it) is an object.

- In C:

```
typedef struct complex {  
    double real;  
    double imag;  
} Complex;  
  
//We create 2 variables of type Complex  
Complex c1,c2;  
c1.real=5;c1.imag=-3; //c1 is 5-3i  
c2.real=4;c2.imag=8; //c2 is 3+8i
```

- In Java:

```
class Complex {  
    double real;  
    double imag;  
}  
  
//Elsewhere in the code:  
//We create 2 OBJECTS of the class Complex  
Complex c1=new Complex();  
Complex c2=new Complex();  
c1.real=5;c1.imag=-3; //c1 is 5-3i  
c2.real=4;c2.imag=8; //c2 is 3+8i
```

- On the right side, we created a **class** (Complex) with **2 objects** (c1,c2)
- An object is created by running “new <class name>()”. It creates the memory space to store the 2 internal variables.

Object Oriented Programming: Functions in classes

- In C: The function needs to get the complex number to show its values.
- In Java: function inside the class. No parameter is needed since accessing to internal variables of the class.

```
typedef struct complex {  
    double real;  
    double imag;  
} Complex;  
  
void showComplex(Complex c1) {  
    if (c1.imag >= 0)  
        printf("%f+%fi\n", c1.real, c1.imag);  
    else  
        printf("%f%f\n", c1.real, c1.imag);  
}  
//We create 2 variables of type Complex  
Complex c1;  
c1.real = 5; c1.imag = -3; //c1 is 5-3i  
showComplex(c1);
```

```
class Complex {  
    double real;  
    double imag;  
  
    //internal function  
    void show() {  
        if (imag >= 0)  
            printf("%f+%fi\n", real, imag); //internal variables  
        else  
            printf("%f%f\n", real, imag);  
        }  
    }  
  
    //Elsewhere in the code:  
    Complex c1 = new Complex();  
    c1.real = 5; c1.imag = -3; //c1 is 5-3i  
    c1.show(); //call to an internal function
```

Object Oriented Programming: Constructor

- In C: Initializer function needs all the parameters.
- In Java: A constructor is a function that is called when an object is created and it allows the initialization of the internal variables (atributes).

```
typedef struct complex {
    double real;
    double imag;
} Complex;

void initComplex(Complex *c1,double real,double imag) {
    c1->real=real;
    c1->imag=imag;
}

void showComplex(Complex c1) {
    if (c1.imag>=0)
        printf("%f+%fi\n",c1.real,c1.imag);
    else
        printf("%f%fi\n",c1.real,c1.imag);
}

//We create 2 variables of type Complex
Complex c1;
initComplex(&c1,5,-3); //c1 is 5-3i
showComplex(c1);
```

```
class Complex {
    double real;
    double imag;

    //Constructor: has the same name than the class and nothing is typed on the left of the name.
    Complex(double r, double i) {
        real=r;
        imag=i;
    }

    //internal function
    void show() {
        if (imag>=0)
            System.out.println (real+"+"+imag+"i"); //internal variables
        else
            System.out.println (real+"-"+imag+"i");
    }
} //end of class

//Elsewhere in the code:
Complex c1=new Complex(5,-3); //It calls the constructor. c1 is 5-3i
c1.show(); //call to an internal function
```

Object Oriented Programming: Another internal function

- An example on how to create another kind of internal function.
 - In C:

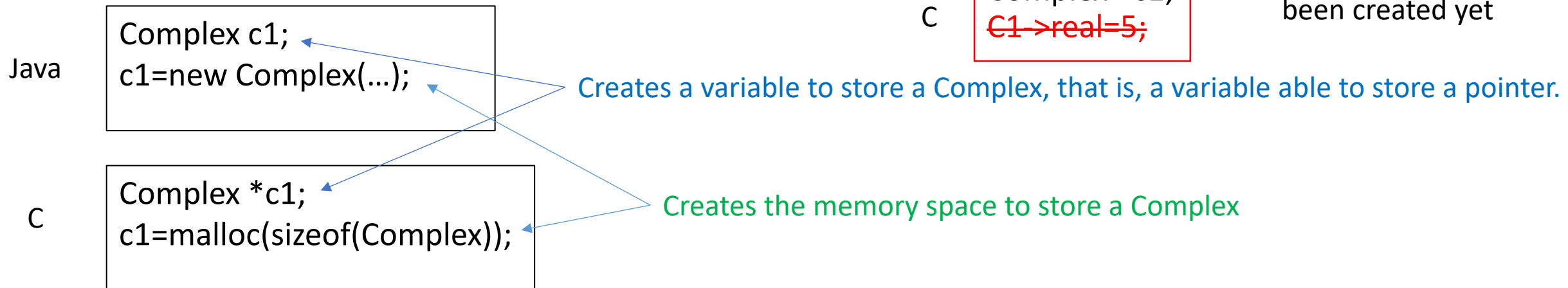
```
typedef struct complex {  
    double real;  
    double imag;  
} Complex;  
//Add c2 to c1 and store the result on c1  
void add(Complex *c1, Complex c2) {  
    c1->real=c1->real+c2.real;  
    c1->imag=c1->imag+c2.imag;  
}  
Complex c1,c2;  
c1.real=5;c1.imag=-3; //c1 is 5-3i  
c2.real=4;c2.imag=8; //c2 is 3+8i  
add(&c1,c2); //adds c2 to c1
```

- In Java:

```
class Complex {  
    double real;  
    double imag;  
  
    Complex(double r, double i) {  
        real=r;  
        imag=i;  
    }  
    void add(Complex c1) {  
        real=real+c1.real;  
        imag=imag+c1.imag;  
    }  
}  
  
//Elsewhere in the code:  
//We create 2 OBJECTS of the class Complex  
Complex c1=new Complex(5,-3);  
Complex c2=new Complex(4,8);  
c1.add(c2); //adds c2 to c1
```

Classes and Objects: analogy with C

- An object is an instance of a class.



- Both codes are equivalent.
- So, in Java a variable that stores an object, stores the pointer to the object (the memory address where the object is allocated), as also does in C.

Passing objects to functions

- As the variable that stores an object is, in fact, a pointer, objects are passed by reference to functions:

- In C:

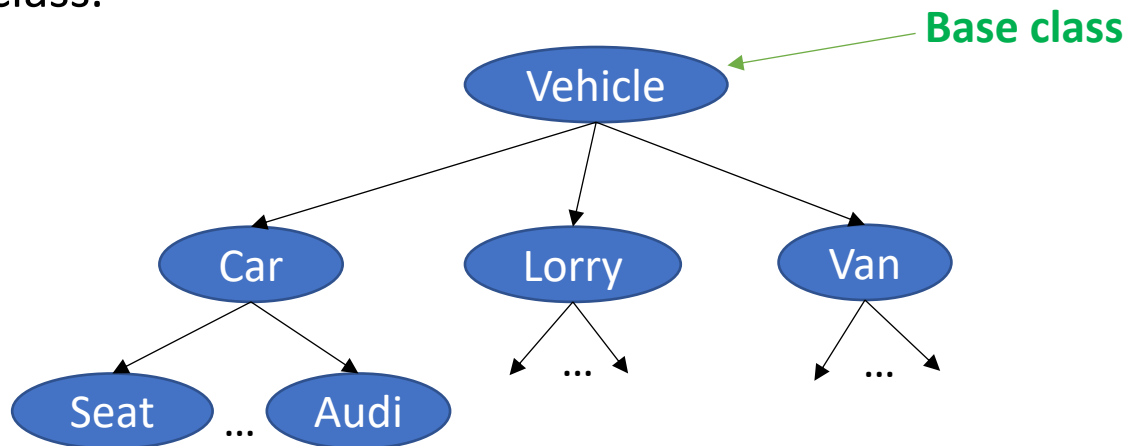
```
typedef struct complex {  
    double real;  
    double imag;  
} Complex;  
//Add c to both fields of c1  
void addConstant(Complex *c1,double c) {  
    c1->real=c1->real+c;  
    c1->imag=c1->imag+c;  
}  
Complex c1;  
c1.real=5;c1.imag=-3; //c1 is 5-3i  
addContant(&c1,10);  
printf("%f , %fi \n",c1.real,c1.imag);
```

- In Java:

```
class Complex {  
    double real;  
    double imag;  
  
    Complex(double r, double i) {  
        real=r;  
        imag=i;  
    }  
    //This function could be defined in another class  
    void addComplex(Complex c1,double c) {  
        c1.real=c1.real+c;  
        c1.imag=c1.imag+c;  
    }  
}  
  
//Elsewhere in the code:  
Complex c1=new Complex();  
c1.real=5;c1.imag=-3; //c1 is 5-3i  
c1.addContant(c1,10);  
printf("%f , %fi \n",c1.real,c1.imag);
```

Object Oriented Programming: inheritance

- A class can derive from another one: parent class and child class.
 - In java, a class can only have 1 parent class (single inheritance).
 - C++ and Python allow multiple inheritance
- If a class derives from another one, the child class inherits the features (variables and functions) of the parent class.
- So, the child class incorporates the variables and functions of the parent child automatically. And not only from the parent class, everything in all its ancestor classes are incorporated to the child class.

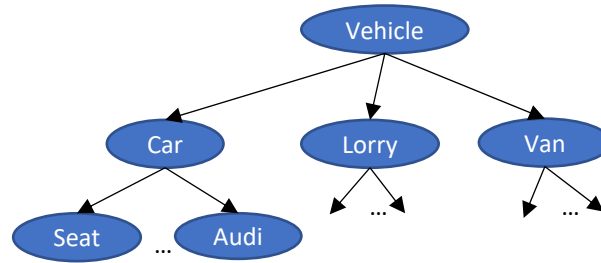


Seat derives directly from **Car** and indirectly from **Vehicle**, so an object of **Seat** will have available all its variables and functions and also those from **Car** and **Vehicle**.

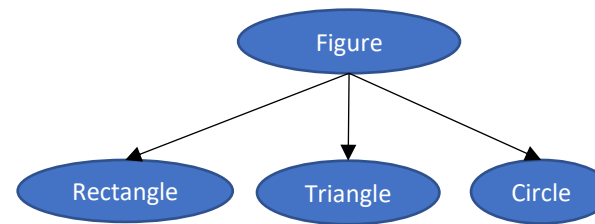
Top class (**Vehicle**) is named **Base class**.

Object Oriented Programming: inheritance

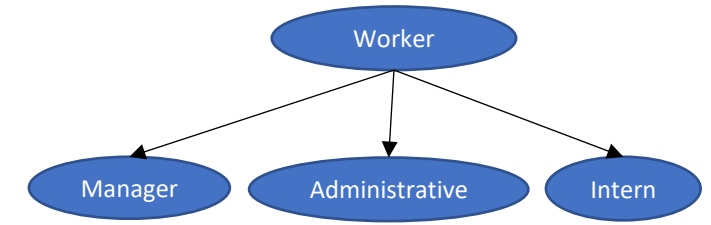
- The more to the top, the more generic the class is.
- When designing an OOP application, top classes share features with their derivative classes. Examples:



An object of the class Seat is a Seat and also a Car and a Vehicle.



An object of the class Triangle is a Triangle and also a Figure.



An object of the class Manager is a Manager and also a Worker.

Object Oriented Programming: inheritance

- An example of inheritance in classes:

```
class Figure {           //Definition of the class Figure
    int x,y;
}
class Rectangle extends Figure {           //Rectangle inherits from Figure
    int width,height;                       //In Rectangle only width,height are declared. x,y are inherited from Figure
    Rectangle(int x1,int y1,int w1, int h1) {
        x=x1;y=y1;width=w1;height=h1;     //Rectangle inherits x,y from Figure. So Rectangle has 4 variables
    }
    void printRectangle() {
        System.out.println("x="+x+" , "y="+y+" , "width="+width+" , "height="+height );
    }
}
class Main {
    public static void main(String argv[]) {
        Rectangle r1=new Rectangle(1,2,3,4);
        r1.printRectangle();
    }
}
```

Java: static variables and functions

- **static variables:** are common to all the objects of the class.

```
class Point {  
    static int numPoints=0; //Executed before creating the 1st object  
    int x,y;  
    Point(int x1,int y1) { //Constructor  
        x=x1;y=y1;  
        numPoints++; //For every Point created, counter is incremented  
    }  
}  
class Main {  
    public static void main(String argv[]) {  
        Point p1=new Point(1,2);  
        Point p2=new Point(3,4);  
        System.out.println(p1.numPoints); //Prints 2  
        System.out.println(p2.numPoints); //Prints 2  
        System.out.println(Point.numPoints); //Accessed also from the class name  
    }  
}
```

- **Static functions:** can only work with static variables.

```
class Point {  
    static int numPoints=0; //Executed before creating the 1st object  
    int x,y; //static function getNumPoints cannot see x,y variables since they're not static  
    Point(int x1,int y1) {  
        x=x1;y=y1;  
        numPoints++;  
    }  
    static int getNumPoints() { //static functions can only work with static variables  
        return numPoints;  
    }  
}  
class Main {  
    public static void main(String argv[]) {  
        Point p1=new Point(1,2);  
        Point p2=new Point(3,4);  
        System.out.println( p1.getNumPoints() ); //Prints 2  
        System.out.println( Point.getNumPoints() ); //Accessed also from the class name  
    }  
}
```

- static elements are usually considered as variables and functions that should be outside the class but they are not able due to Java restrictions.

Java: Main packages

- There are many packages:
 - java.lang: Main classes of Java, related with the language:
 - String, System, classes for data types (Integer, Double, etc.), etc.
 - java.io: Input/Output classes
 - java.util: Classes of utilities:
 - Random, Lists, Date, etc.
 - java.awt/javax.swing:
 - Classes for graphics

Java: Exceptions

- Exceptions allow to control the errors. Use:

- With throws:

```
class Main {  
    public static void main(String argv[])  
        throws FileNotFoundException {  
        FileInputStream f1=new FileInputStream("f1.txt");  
        Scanner s=new scanner(f1);  
        String line=s.nextLine();  
        s.close();  
    }  
}
```

Java treats the error.

Developer treats the error.

- With try..catch:

```
class Main {  
    public static void main(String argv[]) {  
        try {  
            FileInputStream f1=new FileInputStream("f1.txt");  
            Scanner s=new scanner(f1);  
            String line=s.nextLine();  
            s.close();  
        } catch (FileNotFoundException e) {  
            System.out.println("Error: File not found");  
        }  
    }  
}
```

Java: Interfaces

- An **interface** is a **collection of constants and function headers**.
- An interface is commonly used to be added to a class. Then we say that a particular class **implements** an interface.
- In Java, a class can only derive from 1 class, but may implement n interfaces.
- If a class implements an interface, that class:
 - Incorporates all the constants defined in the interface.
 - Must define code for all the functions defined in the interface.

An interface ensures that the class has some functions implemented, ready to be called from another part of the code.

```
interface I1 {  
    double half(double x);  
    double power2(double x);  
}  
  
class Example implements I1 {  
    //define constructor, functions attributes, etc.  
    double half(double x) { //obligated to define code for this function  
        return x/2;  
    }  
    double power2(double x) { //obligated to define code for this function  
        return x*x;  
    }  
}
```

Java: this & super

- **this**: pointer to the current object

```
class Point{
    double x,y;
    Point(double x,double y) {
        this.x=x;
        this.y=y;
    }
}
```

It allows to use the same name of variables for both the parameters and the class attributes.

- **super**: points to the parent class:
 - Call the constructor of the parent class (super with brackets)
 - Call a function of the parent class (super with dot ".")

```
class Figure {
    double x,y;
    Figure(double x,double y) {
        this.x=x;
        this.y=y;
    }
    void printValues() {
        System.out.println("x="+x+" y="+y);
    }
}

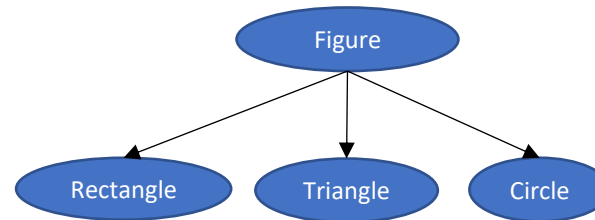
class Rectangle extends Figure {
    double width,height;
    Rectangle(double x, double y, double width ,double height) {
        super(x,y); //calls the constructor function of the parent class
        this.width=width;
        this.height=height;
    }
    void printValues() {
        super.printValues(); //calls to printValues of Figure
        System.out.println("width="+width+" height="+height);
    }
}
```

Java: abstract classes and functions

- A class is called an Abstract class if it contains one or more abstract methods.
- An abstract method is a method that is declared, but contains no implementation.
- Abstract classes cannot be instantiated, and its abstract methods must be implemented by its subclasses

Java: abstract classes and functions

```
abstract class Figure {  
    double x,y;  
    Figure(double x,double y) {  
        this.x=x;  
        this.y=y;  
    }  
    abstract double area();  
}
```



```
class Rectangle extends Figure {  
    double width,height;  
    Rectangle(double x, double y, double width ,double height) {  
        super(x,y); //calls the constructor function of the parent class  
        this.width=width; this.height=height;  
    }  
    double area() { //code definition of the abstract class  
        return width*height;  
    }  
}
```

```
class Circle extends Figure {  
    double radius;  
    Circle(double x, double y, double radius) {  
        super(x,y); //calls the constructor function of the parent class  
        this.radius=radius;  
    }  
    double area() { //code definition of the abstract class  
        return Math.PI*radius*radius;  
    }  
}
```

Java: abstract

- **Without** the abstract function:

```
Figure v[]=new Figure[3];
v[0]=new Circle(10,10,2);
v[1]=new Rectangle(50,10,2,4);
v[2]=new Rectangle(10,50,3);

for(int i=0;i<v.length;i++) {
    if (v[i] instanceof Circle)
        double a=( (Circle)v[i] ).area(); //run area of Circle
    else
        if (v[i] instanceof Rectangle)
            double a=( (Rectangle)v[i] ).area(); //run area of Rect.
}
```

- We need to check and cast every object.
- **instanceof** operator allows to ask if an object is an instance of a class.

- **With** the abstract function:

```
Figure v[]=new Figure[3];
v[0]=new Circle(10,10,2);
v[1]=new Rectangle(50,10,2,4);
v[2]=new Rectangle(10,50,3);

for(int i=0;i<v.length;i++) {
    double a=v[i].area();
}
```

- We **don't** need to check and cast every object. Java do it internally.

Java: Generics

- Available from Java Version 1.5 (Java 5)
- It allows to define the type (class) of a variable when creating the object:

In our Stack class, we could define a class to store any type of data:

```
class Stack {  
    double v[50];  
    int top;  
    Stack () { top=-1;}  
    void push(double value) { .... }  
    double pop() { .... }  
}
```



```
class Stack<T> {  
    T v[50];  
    int top;  
    Stack<T> () { top=-1;}  
    void push(T value) { .... }  
    T pop() { .... }  
}
```

Before 1.5 version, ArrayList stored objects of class Object (to be able to store objects of any existing class). To reach the intValue() function of Integer, needed to cast from Object to Integer.

```
ArrayList v=new ArrayList();  
v.add( new Integer(10) );  
v.add( new Integer(20) );  
v.add( new Integer(30) );  
  
for(int i=0;i<v.size();i++) {  
    System.out.println( ((Integer)v.get(i)).intValue() );  
}
```

ArrayList functions:

- **add(object)**: adds element
- **size()**->length
- **get(index)** -> value at "index" position, starting by 0

After 1.5 version, ArrayList stored objects of class Integer. So, we can directly call the intValue() function of Integer, without needing to cast.

```
ArrayList<Integer> v=new ArrayList<Integer>();  
v.add( new Integer(10) );  
v.add( new Integer(20) );  
v.add( new Integer(30) );  
  
for(int i=0;i<v.size();i++) {  
    System.out.println( v.get(i).intValue() );  
}
```

Java: autoboxing and foreach

- Available from Java Version 1.5 (Java 5)
- **Autoboxing**
 - Java automatically converts from classes to values and vice-versa when needed and without needing to call specific functions. Java does it implicitly
 - It only works in classes associated to data types (in java.lang): Integer, Double, etc.

```
ArrayList<Integer> v=new ArrayList<Integer>();
v.add( 10 ); //we don't need new Integer(10)
v.add( 20 );
v.add( 30 );

int sum=0;
for(int i=0;i<v.length;i++) {
    sum+=v.get(i); //converts Integer to int automatically
}
System.out.println("Sum:"+sum);
```

- **Foreach**
 - Shorter for expression when iterating a collection: array, ArrayList, etc.

```
ArrayList<Integer> v=new ArrayList<Integer>();
v.add( 10 );
v.add( 20 );
v.add( 30 );

for(int n:v) { //foreach integer n in the arraylist v
    System.out.println( n );
}
```

```
double v[]={10,20,30,40,50};

for(double d:v) { //foreach integer n in the arraylist v
    System.out.println( d );
}
```

Java: packages

- Classes, interfaces and exceptions are grouped into packages.
- Every class belongs to a package. If no package is defined, then it belongs to the **default package**.
- Packages are just a way to organize classes by topics.
- A class belongs to a package iif:
 - The file is inside a folder with the name of the package.
 - First line of the file contains the sentence:
`package <package_name>;`

Java: Visibility

- Classes and their functions and attributes may have modifiers that affect their visibility by other classes and functions. These modifiers are:
 - **private**: the function or attribute is only visible from functions of the same class.
 - **package (blank)**: The class, function or attribute is only visible from classes of the same package.
 - **public**: The class, function or attribute is visible from all the classes of all the packages.
 - **protected**: Allows visibility from derived classes of different package

Java: JAR file

- A JAR file is like a zip file for Java.
- It allows to pack several .class files into a unique file, compressed and ready to be used in applications without uncompressing it.
- A JAR file can also be executable, that is, containing the whole application and running it just double-clicking on the icon.